

PROBLEM SHEET 1

Alex Kavvos

1. Write down a derivation of the judgement

$$\text{succ}(\text{succ}(\text{succ}(\text{zero}))) \text{ odd}$$

2. (i) Write down the rules that generate lists of natural numbers.
 (ii) Write down the associated induction principle.
 (iii) In your notation, write a derivation of the judgement that $[\mathbf{0}, \mathbf{1}]$ is a list.
3. (*) All the judgements we have seen up to this point have been *unary*, in the sense that they referred to only one entity. For example, the judgement $n \text{ nat}$ only refers to the object n .

However, judgements can have arbitrary *arity*, and can thus define arbitrary relations between an arbitrary number of objects. For example, the following *ternary* judgment $\text{sum}(a, b, c)$ defines a relation between three objects: a , b and c .

$$\frac{\text{BASE} \quad b \text{ nat}}{\text{sum}(\text{zero}, b, b)}$$

$$\frac{\text{IND} \quad \text{sum}(a, b, c)}{\text{sum}(\text{succ}(a), b, \text{succ}(c))}$$

The judgement $\text{sum}(a, b, c)$ can be written in more familiar notation as $a + b = c$.

Such judgements can be used—amongst countless other things—to define functions. This exercise is about showing that the above rules define the addition function.

- (i) Write down a derivation of $\text{sum}(\text{succ}(\text{zero}), \text{succ}(\text{zero}), \text{succ}(\text{succ}(\text{zero})))$.
- (ii) Restate the above rules as a Haskell function on the data type

```
data Nat = Zero | Succ Nat
```

Does your code use pattern matching? Discuss its relation to the rules given above.

- (iii) Prove that if $\text{sum}(a, b, c)$ then $a \text{ nat}$, $b \text{ nat}$, and $c \text{ nat}$.
- (iv) (Existence) Prove that if $a \text{ nat}$ and $b \text{ nat}$ then there exists a $c \text{ nat}$ such that $\text{sum}(a, b, c)$.
- (v) (Uniqueness) Prove that if $\text{sum}(a, b, c)$ and $\text{sum}(a, b, c')$ it must be that $c = c'$.
- (vi) Conclude that $\text{sum}(a, b, c)$ indeed defines a function on natural numbers.